BTS SIO · 2E ANNÉE · DÉVELOPPEMENT WEB

JavaScript et le DOM

Rappels, puis stockage local, JSON et fetch

```
document.querySelector("#app")
```

PROGRAMME · 2 H 30

Un fil rouge : une liste de tâches qui grandit

Partie 1 · Rappels DOM

Sélectionner, modifier, réagir, créer

App. 1 · une fiche à modifier

App. 2 · la liste de tâches

Partie 2 · Données

Tableaux, localStorage et JSON

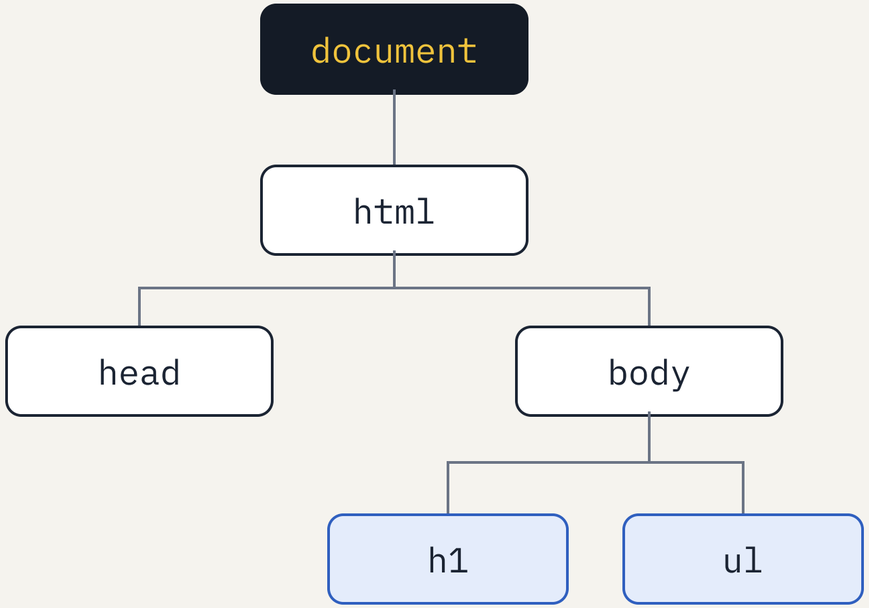
App. 3 · des tâches qui survivent au rechargement

Partie 3 · Aller plus loin

Délégation, fetch et async / await

App. 4 · afficher des données d'une API

Le navigateur transforme le HTML en arbre



Chaque balise devient un **nœud** que JavaScript peut lire, modifier, ajouter ou supprimer.

On relie le script dans le **head** avec l'attribut **defer** : il attend que l'arbre soit prêt.

```
<script src="app.js" defer></script>
```

querySelector parle le langage du CSS

```
// Le premier élément trouvé, ou null
const titre = document.querySelector("h1");
const form = document.querySelector("#ajout");
const lien = document.querySelector("nav a.actif");

// Tous les éléments : une NodeList
const items = document.querySelectorAll(".tache");
items.forEach(item => console.log(item));
```

querySelector

le premier élément, ou **null** si rien ne correspond

querySelectorAll

une liste, qu'on parcourt avec forEach

getElementById

l'ancienne façon, toujours valable : sans le #

Une fois sélectionné, l’élément se manipule

Propriété ou méthode	Ce qu’elle touche	Exemple
`el.textContent`	le texte	titre.textContent = "Bonjour"
`el.innerHTML`	le HTML intérieur	à éviter avec du texte saisi
`input.value`	la saisie d’un champ	const nom = champ.value
`el.classList.add / remove / toggle`	les classes CSS	tache.classList.toggle("faite")
`el.setAttribute(nom, val)`	un attribut	img.setAttribute("alt", "Logo")
`el.dataset`	les attributs data-*	li.dataset.id → data-id
`el.style.propriete`	un style en ligne	à réserver aux cas dynamiques

Bonne pratique : on change l’apparence en ajoutant ou retirant une **classe**, et on laisse le style dans le CSS.

APPLICATION 1 · 15 MIN

Modifier une fiche profil, sans toucher au HTML

Le HTML de départ

```html
<article class="profil" data-role="etudiant">
  <h2>Prénom Nom</h2>
  <p class="option">Option</p>
  <ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>PHP</li>
  </ul>
</article>
```

Dans app.js uniquement

1. mettez votre nom dans le h2
2. écrivez « SLAM » ou « SISR » dans .option
3. ajoutez la classe « mise-en-avant » sur la fiche
4. affichez dans la console le data-role
5. mettez en gras toutes les compétences avec querySelectorAll
6. Bonus : comptez les compétences et affichez le total

addEventListener : quand il se passe X, fais Y

```
const form = document.querySelector("#ajout");
const champ = document.querySelector("#titre");

form.addEventListener("submit", (event) => {
  event.preventDefault();  // pas de rechargement
  const texte = champ.value.trim();
  if (texte === "") return;
  console.log("Nouvelle tâche :", texte);
  form.reset();
});
```

Les événements les plus utiles

Événement	Quand
`click`	clic sur l'élément
`input`	à chaque frappe
`change`	case, liste, valeur validée
`submit`	envoi d'un formulaire
`keydown`	touche du clavier

Fabriquer un élément, puis l’accrocher à la page

```
const liste = document.querySelector("#taches");

// 1. Créer
const li = document.createElement("li");
li.textContent = texte;
li.classList.add("tache");

// 2. Accrocher
liste.append(li);

// 3. Supprimer
li.remove();
```

createElement

crée l’élément en mémoire : il n’est pas encore visible

append / prepend

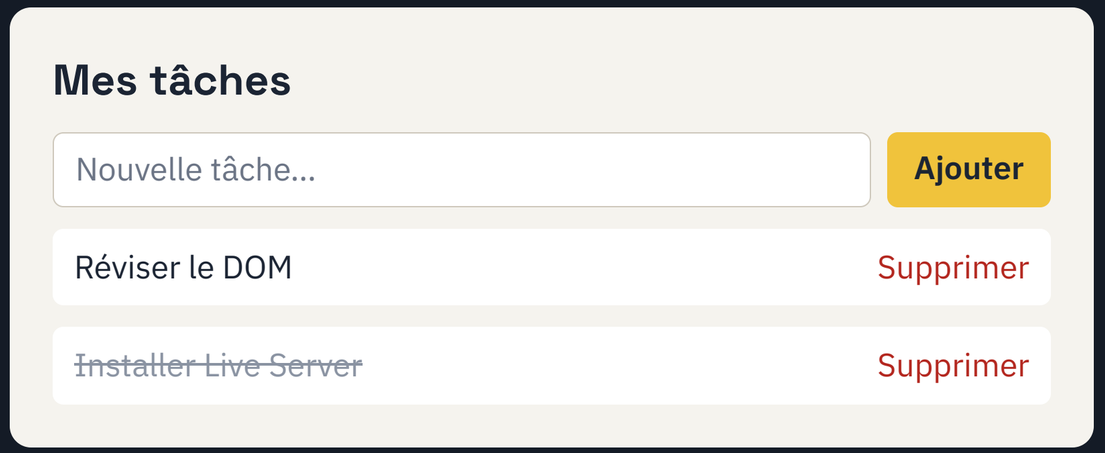
l’ajoute à la fin, ou au début, d’un parent

remove

le retire de la page

APPLICATION 2 · 25 MIN

La liste de tâches, version 1



Une tâche barrée = une tâche terminée

1. HTML : un form avec un champ et un bouton, une ul vide
2. à l'envoi du formulaire, créez un li avec le texte saisi
3. refusez les tâches vides et videz le champ après l'ajout
4. un clic sur la tâche la barre ou la débarre (classe « faite »)
5. un bouton « Supprimer » dans chaque li la retire
6. Bonus : affichez « 3 tâches restantes » en temps réel

PARTIE 2 · LES DONNÉES

Rechargez la page : vos tâches ont disparu.

Pour qu’elles restent, il faut séparer les données de l’affichage, puis les stocker.

Un tableau d'objets, et une fonction qui affiche

```
let taches = [
  { id: 1, titre: "Réviser", faite: false },
  { id: 2, titre: "Coder", faite: true },
];

function afficher() {
  liste.textContent = "";  // on vide
  taches.forEach(tache => {
    const li = document.createElement("li");
    li.textContent = tache.titre;
    li.classList.toggle("faite", tache.faite);
    liste.append(li);
  });
}
```

La vérité est dans le **tableau**, pas dans la page.

Pour ajouter, cocher ou supprimer : on modifie le tableau, puis on rappelle **afficher()**.

```
taches.push(…)  ajouter
taches.filter(…) garder certaines
taches.find(…)  en trouver une
```

Un petit espace de stockage dans le navigateur

```
// Enregistrer : une clé, une valeur
localStorage.setItem("theme", "sombre");

// Lire : null si la clé n'existe pas
const theme = localStorage.getItem("theme");

// Supprimer une clé, ou tout
localStorage.removeItem("theme");
localStorage.clear();
```

Persistant

survit au rechargement et à la fermeture du navigateur

Par site

chaque site a le sien, sur chaque navigateur

Que du texte

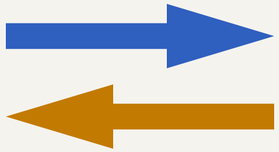
les valeurs sont toujours stockées en chaînes de caractères

JSON transforme un tableau en texte, et inversement

Objet JavaScript

```
[{ id: 1, titre: "Coder" }]
```

JSON.stringify



JSON.parse

Chaîne de caractères

```
'[{"id":1,"titre":"Coder"}]'
```

```
localStorage.setItem("taches", JSON.stringify(taches));
taches = JSON.parse(localStorage.getItem("taches"));
```

Charger au démarrage, sauvegarder à chaque changement

```
// Au démarrage : [] si rien n'est encore stocké
let taches = JSON.parse(localStorage.getItem("taches")) ?? [];

function sauvegarder() {
  localStorage.setItem("taches", JSON.stringify(taches));
}

function ajouter(titre) {
  taches.push({ id: Date.now(), titre, faite: false });
  sauvegarder();
  afficher();
}

afficher();  // premier affichage
```

1. On modifie le tableau

2. On sauvegarde

3. On réaffiche

Toujours dans cet ordre, pour chaque action : ajouter, cocher, supprimer.

Trois façons de garder une information côté navigateur

	localStorage	sessionStorage	Cookie
Durée	jusqu’à suppression	le temps de l’onglet	date d’expiration choisie
Envoyé au serveur	non	non	oui, à chaque requête
Taille	environ 5 Mo	environ 5 Mo	environ 4 Ko
Lu par	le JS de la page	le JS de l’onglet	le serveur, et le JS sauf si HttpOnly
Usage typique	préférences, brouillon, panier	étapes d’un formulaire	session de connexion

sessionStorage s’utilise exactement comme localStorage : mêmes méthodes, seule la durée change.

APPLICATION 3 · 25 MIN

Des tâches qui survivent au rechargement

Reprenez votre liste de l’application 2

1. créez le tableau taches et la fonction afficher()
2. réécrivez l’ajout : on modifie le tableau, puis on réaffiche
3. faites de même pour cocher et supprimer, grâce à un data-id sur chaque li
4. sauvegardez dans le localStorage après chaque changement
5. chargez les tâches au démarrage : rechargez la page pour tester
6. Bonus : trois boutons de filtre, Toutes, À faire, Terminées

C’est réussi si…

- F5 ne fait plus rien disparaître
- la clé « taches » est visible dans les DevTools, onglet Application
- aucune modification de la page sans passer par le tableau

Un seul écouteur sur le parent, au lieu d'un par élément

```
liste.addEventListener("click", (event) => {
  const li = event.target.closest("li");
  if (!li) return;  // clic hors d'une tâche
  const id = Number(li.dataset.id);

  if (event.target.matches(".supprimer")) {
    taches = taches.filter(t => t.id !== id);
  } else {
    const t = taches.find(t => t.id === id);
    t.faite = !t.faite;
  }
  sauvegarder();
  afficher();
});
```

Un clic sur un bouton **remonte** jusqu'à ses parents : la liste l'entend aussi.

event.target

l'élément vraiment cliqué

closest(sélecteur)

remonte jusqu'au premier parent qui correspond

Ça marche même pour les éléments créés **après** coup.

Récupérer des données sur un serveur, sans recharger la page

```
async function chargerUtilisateurs() {
  try {
    const reponse = await fetch("https://jsonplaceholder
      .typicode.com/users");
    if (!reponse.ok) {
      throw new Error("Erreur " + reponse.status);
    }
    const users = await reponse.json();
    console.log(users);
  } catch (erreur) {
    console.error(erreur);
  }
}
```

async / await

attendre la réponse sans bloquer la page

reponse.ok

faux si le serveur répond 404 ou 500 : fetch ne lève pas d’erreur tout seul

reponse.json()

le JSON reçu devient un tableau d’objets

Une page qui charge a trois états à prévoir

Chargement

« Chargement… » affiché avant le fetch

Succès

on vide, puis on crée un élément par objet

Erreur

un message lisible dans la page, pas seulement dans la console

```
statut.textContent = "Chargement…";
users.forEach(user => {  /* createElement, textContent, append */  });
statut.textContent = "";  // ou, dans le catch : "Impossible de charger les données"
```

APPLICATION 4 · 25 MIN

Un annuaire alimenté par une API

Nouveau projet · source : jsonplaceholder.typicode.com/users

1. au chargement, récupérez les utilisateurs avec fetch
2. affichez une carte par personne : nom, e-mail, ville
3. gérez les états chargement et erreur dans la page
4. ajoutez un champ de recherche qui filtre les cartes à chaque frappe
5. un clic sur une carte l'ajoute aux favoris, mémorisés dans le localStorage
6. Bonus : un bouton « Afficher seulement mes favoris »

Ce qu'on réutilise

- fetch, async / await, try / catch
- un tableau et une fonction afficher()
- l'événement input et filter()
- la délégation pour les clics
- localStorage et JSON pour les favoris

SÉCURITÉ · À RETENIR

Deux erreurs qui ouvrent la porte à une attaque

innerHTML avec du texte saisi

```
li.innerHTML = champ.value;  // faille XSS
li.textContent = champ.value; // sûr
```

Si l'utilisateur tape du HTML ou un script, innerHTML l'exécute. textContent l'affiche comme du simple texte.

Des données sensibles dans le localStorage

Tout script exécuté sur la page peut lire le localStorage, y compris un script injecté.

On n'y met **jamais** de mot de passe, de jeton de connexion ou de donnée personnelle.

Préférences, brouillons, favoris : oui.

RÉCAPITULATIF

Ce qu’il faut retenir aujourd’hui

01 querySelector parle CSS, et defer attend le DOM

02 classList plutôt que style

03 submit + preventDefault sur un formulaire

04 modifier le tableau, sauvegarder, réafficher

05 localStorage ne stocke que du texte

06 JSON.stringify pour stocker, JSON.parse pour relire

07 un seul écouteur sur le parent, avec closest

08 fetch : vérifier reponse.ok, prévoir l’erreur

Et toujours : textContent pour afficher ce que l’utilisateur a saisi.